

Architecture & Production Design

FrontierAtlas Ingestion Pipeline — Phase VI design document

Stack: Python 3.11+ · asyncio + aiohttp · BeautifulSoup/lxml · feedparser · Playwright (async, escalation only) · rapidfuzz · Gemini Flash → Groq Llama 3 → DeepSeek (REST) · JSONL data lake → Google Sheets export.

1. Scale Strategy — 500,000+ records without manual intervention

Principle: the unit of work is a URL, and every component is stateless against a queue.

```
[Seed/Discovery] → [URL Frontier (queue)] → [N crawler workers] → [Raw store]
                                     → [M LLM extraction workers] → [Resolver] → [Sink]
```

- **Discovery is decoupled from extraction.** Directory/index/sitemap crawlers only emit URLs into a frontier queue; detail-page workers only consume. Reaching 500k = raising worker count and queue depth — no code changes.
- **Bulk sources are paginated APIs where possible** (arXiv API, Hugging Face Papers API, public startup directories). API pagination is resumable by cursor, so a crashed worker restarts from its checkpoint, not from zero.
- **Backpressure, not buffering:** a bounded `asyncio.Semaphore` global cap plus per-domain caps keep memory flat regardless of frontier size; the queue absorbs bursts.
- **Idempotency everywhere:** every record write is keyed by URL fingerprint. Re-running any stage is safe, which is what makes unattended operation possible — the recovery strategy for every failure class is simply "retry the unit".
- **Single-node → distributed with config only:** the in-process frontier and seen-set are interfaces; swapping `asyncio.Queue` → Redis/SQS and `set()` → Redis SET is a config change, not a redesign.

2. Handling 413s & 429s across thousands of concurrent extractions

413 / context overflow — never send an oversized payload:

1. HTML is reduced to semantically dense text first (scripts/nav/boilerplate stripped via lxml).
2. Token count is *estimated client-side* (chars/4 heuristic with a safety margin) against the target model's context budget.
3. Oversized text is truncated **head+tail-biased**: title, meta, and lead paragraphs are always retained; the middle is dropped first (news/job pages carry their signal up front).
4. Budget check runs *per provider* in the fallback chain, since each tier has a different window.

429 / rate limits — provider-aware pacing plus reactive backoff:

- **Proactive:** a token-bucket limiter per provider keyed to its published RPM/TPM, so thousands of concurrent tasks are smoothed *before* hitting the API.
- **Reactive:** on 429, exponential backoff with **full jitter** (`sleep = uniform(0, min(cap, base*2^attempt))`), honoring `Retry-After` when present. Jitter prevents synchronized retry stampedes across workers.
- **Fallback chain as load-shedding:** repeated 429s on tier 1 (Gemini Flash) trip a short-lived circuit breaker that routes traffic to tier 2 (Groq) / tier 3 (DeepSeek) instead of queueing indefinitely — the chain is both a reliability and a throughput mechanism. The provider that produced each record is stamped on the record for audit.

3. Freshness Tracking — never process the same article/job twice, across nodes

- **Canonical URL fingerprint:** URLs are normalized (lowercase host, tracking params stripped, trailing slash collapsed) and hashed (SHA-256). This fingerprint is the global dedup key.
- **Shared seen-set:** single-node runs use a persistent local set (JSON/SQLite); distributed runs point the same interface at Redis `SET NX` — an atomic check-and-claim, so two nodes can never both win the same URL. Claims carry a TTL lease so a crashed worker's URL is re-claimable.
- **24-hour gate before LLM spend:** publication timestamps are extracted from meta tags (`article:published_time`, JSON-LD), RSS `pubDate`, or visible relative dates ("2 hours ago") normalized to UTC ISO-8601. Anything older than 24h is dropped pre-extraction — freshness enforcement costs zero LLM tokens.
- **No-date heuristic:** sources without a trustworthy date fall back to first-seen semantics — content hash of the extracted text; an unseen hash within the monitoring window is treated as new, and the decision is logged on the record (`dateConfidence: "first_seen"`).

4. Storage Strategy

Layer	Choice	Justification
Raw + canonical records	PostgreSQL (JSONB)	Records are semi-structured with a versioned envelope (<code>schemaVersion</code>); JSONB gives schema flexibility with real indexes (GIN on entity name, B-tree on <code>collectedAt</code>), transactional upserts keyed by URL fingerprint, and cheap 500k–50M scale on one primary.
Ingest buffer / trial output	Append-only JSONL	Crash-safe (a partial line is discardable, never corrupting), trivially resumable, streams into both Postgres <code>COPY</code> and the Sheets exporter.
Relationships	Neo4j (graph)	The product <i>is</i> an intelligence graph: startup→product, paper→repo, company→job, investor→startup edges with multi-hop queries ("startups whose papers trend on GitHub and are hiring") that are joins-hostile in SQL but native in Cypher.
Similarity / resolution assist	pgvector (in Postgres)	Embedding search for near-duplicate entity names and article dedup beyond exact hashing — kept inside Postgres to avoid operating a separate vector DB until scale demands it.

Flow: JSONL → Postgres (system of record) → projections into Neo4j (edges) and pgvector (embeddings). Each store is a projection of Postgres, so any of them can be rebuilt — one source of truth, no dual-write consistency problem.

Entity resolution, anti-bot escalation, and the concrete source registries are implemented in `src/` and summarized in [README.md](#).